

XCEPTION KAŞ ROI DEEPPFAKE DETECTION

Kodun Ne Yaptığı, Nasıl Çalıştığı ve Deney Akışının Teknik Açıklaması

Raporun kapsamı

Bu doküman, Xception_Eyebrow_Deepfake_Colab_ROBUST_FIXED notebook'unun çalışma mantığını açıklar. Rapor performans sonuçlarını yorumlamaktan çok; verinin nasıl alındığını, modelin nasıl kurulduğunu, transfer learning ve fine-tuning adımlarını, değerlendirme mekanizmasını ve deney çıktılarının nasıl kaydedildiğini açıklar.

Kaynak notebook: Xception_Eyebrow_Deepfake_Colab_ROBUST_FIXED (1).ipynb

1. Yönetici Özeti

Bu notebook, daha önce yüz görüntülerinden çıkarılmış kaş (eyebrow) ROI görüntülerini kullanarak REAL ve FAKE sınıfları arasında ikili sınıflandırma yapan bir deepfake tespit deneyi yürütür. Kod, kaş bölgesini yeniden tespit etmez veya kırpamaz; hazır ROI verisini eğitim girdisi olarak kabul eder.

Model tarafında ImageNet üzerinde önceden eğitilmiş Xception backbone'u kullanılır. Eğitim iki aşamalıdır: ilk aşamada Xception backbone dondurulur ve yalnızca yeni eklenen sınıflandırıcı katmanlar eğitilir; ikinci aşamada Xception'ın son katmanlarının bir bölümü düşük öğrenme oranıyla açılarak fine-tuning yapılır.

Notebook'un 'ROBUST FIXED' olarak adlandırılmasının temel nedeni veri erişim dayanıklılığıdır. Google Drive'daki kaş ROI datasetinin tamamı eğitim öncesinde Colab'ın yerel /content alanına kopyalanır. Böylece epoch sırasında Drive bağlantısının geçici olarak bir dosyayı bulamaması nedeniyle eğitimin çökmesi riski azaltılır. Eksik veya bozuk görüntüler ayrıca doğrulanır, eğitimden çıkarılır ve log dosyasına yazılır.

Kısaca çalışma akışı

Kaş ROI verisi → Drive kontrolü → Local cache → Veri sayımı ve doğrulama → tf.data pipeline → Kaş-özel hafif augmentation → Xception preprocessing → Frozen transfer learning → Fine-tuning → Validation AUC ile model seçimi → Test tahmini → Metrikler ve grafikler → Final model ve deney artefaktlarının kaydı.

2. Kodun Temel Amacı ve Sınırları

2.1. Kod ne işe yarar?

- Hazır kaş ROI görüntülerini REAL=0 ve FAKE=1 etiketleriyle sınıflandırma problemine dönüştürür.
- ImageNet pretrained Xception'ı kaş bölgesindeki deepfake ayırımına adapte eder.
- İlk aşamada transfer learning, ikinci aşamada fine-tuning uygular.
- Validation AUC değerine göre frozen ve fine-tuned modeller arasından en iyi modeli seçer.
- Seçilen modeli daha önce model seçiminde kullanılmayan test setinde değerlendirir.
- Accuracy, balanced accuracy, precision, recall/sensitivity, specificity, F1, ROC-AUC ve confusion matrix gibi metrikleri üretir.
- Confusion matrix, ROC curve ve eğitim eğrileri dahil rapor görsellerini üretir.
- Her çalıştırma için ayrı bir run klasörü oluşturarak model, log, metrik, tahmin ve ortam kayıtlarını arşivler.

2.2. Kod ne yapmaz?

- Ham yüz frame'lerinden kaş bölgesini bulmaz veya kırpamaz.
- Yeni train/validation/test split oluşturmaz; mevcut klasör ayırımını kullanır.
- LBP, HOG, GLCM, Gabor veya Wavelet gibi el yapımı feature'ları ayrıca çıkarmaz.
- Test setine bakarak model seçmez; test seti yalnızca final değerlendirme için kullanılır.
- Drive'daki orijinal kaş ROI dosyalarını değiştirmez.

3. Girdi ve Çıktı Yapısı

3.1. Girdi klasörü

Notebook, Google Drive üzerinde Deney 1 altındaki Kaş klasörünü kaynak olarak kullanır. Beklenen mantıksal yapı:

```
Kaş/
├── fake/
│   ├── train/
│   ├── val/
│   └── test/
└── real/
    ├── train/
    ├── val/
    └── test/
```

Kod dosyaları recursive olarak taradığı için train/val/test altında video bazlı ek alt klasörler bulunması durumunda da görüntüleri bulabilir.

3.2. Çıktı klasörü

```
Sonuçlar/<timestamp>_xception_eyebrow/
├── checkpoints/
├── logs/
├── metrics/
├── predictions/
├── figures/
├── artifacts/
├── config_resolved.yaml
├── requirements_lock.txt
├── environment.json
├── run_summary.json
└── output_manifest.csv
```

Klasör/Dosya	Görevi
checkpoints/	Frozen ve fine-tuned aşamalardaki en iyi .keras modellerini saklar.
logs/	Epoch logları, dataset kopyalama hataları ve geçersiz dosya kayıtlarını saklar.
metrics/	Training summary, final test metrikleri ve classification report gibi ölçümleri saklar.
predictions/	Her test görüntüsü için P(fake), gerçek etiket, tahmin ve doğruluk bilgisini saklar.
figures/	Confusion matrix, ROC curve, eğitim eğrileri ve karşılaştırma grafiklerini saklar.
artifacts/	Nihai seçilen model ile model konfigürasyonunu saklar.
run_summary.json	Tüm deney koşullarını ve sonuç özetini tek dosyada toplar.
output_manifest.csv	Üretilen dosyaların yolunu, boyutunu ve SHA-256 hash bilgisini listeler.

4. Uçtan Uca Çalışma Akışı

Aşama	İşlem	Çıktı
1	Ortamı hazırla	TensorFlow/Keras, Xception, sklearn metrikleri, seed=42
2	Drive'ı bağla ve klasörleri doğrula	Kaş/fake-real/train-val-test yolları kontrol edilir
3	Dataseti local cache'e kopyala	/content/eyebrow_dataset_cache
4	Veriyi say ve doğrula	Sınıf dağılımı + bozuk/eksik dosya logu
5	tf.data pipeline kur	RGB 3 kanal, 224×224, float32, shuffle/batch/prefetch
6	Train augmentation + Xception preprocess	Kaş dokusunu koruyan hafif dönüşümler
7	Xception modelini kur	Frozen pretrained backbone + yeni binary classifier
8	Frozen training	Classifier eğitimi, val_auc ile checkpoint
9	Fine-tuning	Son katmanların açılması, düşük LR
10	Model seçimi	Frozen vs fine-tuned validation AUC karşılaştırması
11	Test tahmini	Her görüntü için P(fake), threshold=0.50
12	Metrik ve görseller	Accuracy, F1, AUC, confusion matrix, ROC ve eğitim eğrileri
13	Final export	Final model, config, environment, summary, manifest

5. Hücre Bazlı Teknik Açıklama

5.1. Cell 1 - Imports and Experiment Setup

İlk hücre deneyin yazılım ortamını ve temel parametrelerini hazırlar. TensorFlow/Keras modelleme için, NumPy/Pandas veri işlemleri için, Matplotlib görselleştirme için ve scikit-learn değerlendirme metrikleri için yüklenir.

Parametre	Değer	Anlamı
SEED	42	Python, NumPy ve TensorFlow rastgeleliğini mümkün olduğunca tekrarlanabilir kılar.
IMG_SIZE	224 × 224	Xception'a verilecek ROI boyutudur.
BATCH_SIZE	16	Bir optimizasyon adımında modele verilen örnek sayısıdır.
Sınıflar	REAL=0, FAKE=1	Pozitif sınıf FAKE olarak ele alınır.

GPU kontrolü de bu hücrede yapılır. GPU bulunmazsa kod çalışabilir ancak eğitim süresi belirgin biçimde uzayabilir.

5.2. Cell 2 - Drive Paths, Local Cache ve Run Yapısı

Bu hücre notebook'un dayanıklılık katmanıdır. Önce Google Drive bağlanır ve deney yolları tanımlanır. Her çalıştırmada zaman damgalı benzersiz bir run klasörü açılır; böylece önceki deneylerin üzerine yazılmaz.

- Kaş klasöründeki fake/real ve train/val/test dizinlerinin tamamı mevcut mu diye kontrol edilir.
- Drive'daki görüntüler /content/eyebrow_dataset_cache dizinine kopyalanır.
- Her dosya için kopyalama en fazla 3 kez denenir.
- Kopyalanamayan dosyalar dataset_copy_failures.csv dosyasına yazılır.
- Eğitim bundan sonra Drive yerine local cache üzerinden yürütülür.
- Kaynak Drive klasöründe hiçbir görüntü değiştirilmez veya silinmez.

Neden local cache?

Model her epoch boyunca görüntülere tekrar erişir. Google Drive bağlantısında anlık bir erişim problemi olursa tf.io.read_file aşamasında NotFoundError oluşabilir. Dataseti önce local /content alanına almak, eğitimin ortasında tek bir dosya erişim hatası yüzünden tüm model.fit çağrısının kesilmesi riskini azaltır.

5.3. Cell 3 - Dataset Count ve Class Balance Check

Bu hücre henüz eğitim yapmaz. Amaç, deney başlamadan önce veri setinin gerçekten beklenen yapıda olup olmadığını kontrol etmektir. Görüntüler .jpg, .jpeg, .png, .bmp ve .webp uzantılarıyla recursive olarak sayılır.

- Fake ve real görüntü sayıları train, validation ve test için ayrı hesaplanır.
- Toplam train/val/test büyüklükleri bulunur.
- Her split için sınıf oranı hesaplanır.
- Dengesizlik belirginse kullanıcı uyarılır.
- Herhangi bir split boşsa eğitim başlatılmadan hata verilir.

5.4. Cell 4 - TensorFlow Data Pipeline

Bu hücre disk üzerindeki dosya yollarını TensorFlow'un verimli tf.data yapısına çevirir. REAL örneklerine 0, FAKE örneklerine 1 etiketi atanır. Dosya sırası kontrollü tutulur; train seti shuffle edilirken validation ve test shuffle edilmez.

Görüntü doğrulama

Model eğitiminden önce her dosya Path.exists() ile varlık kontrolünden ve PIL Image.verify() ile okunabilirlik kontrolünden geçirilir. Hatalı dosyalar eğitim listesinden çıkarılır ve skipped_or_invalid_dataset_files.csv içine yazılır.

Görüntü okuma ve batchleme

1. tf.io.read_file ile dosya binary olarak okunur.
2. tf.io.decode_image ile görüntü 3 kanallı RGB olarak decode edilir.
3. tf.image.resize ile 224×224 boyutuna getirilir.
4. Görüntü ve etiket float32 veri tipine çevrilir.
5. Train verisi seed=42 ile shuffle edilir.
6. Veriler batch_size=16 ile batch'lere ayrılır.
7. tf.data.AUTOTUNE prefetch ile veri hazırlama ve GPU hesaplaması örtüştürülmeye çalışılır.

Önemli tasarım kararı

Test datasetinin shuffle edilmemesi, model.predict çıktılarının test dosya listesiyle birebir aynı sırada kalmasını sağlar. Böylece her tahmin güvenilir biçimde ilgili görüntü dosyasına geri bağlanabilir.

5.5. Cell 5 - Kaş-Özel Augmentation ve Preprocessing

Augmentation yalnızca train verisine uygulanır. Validation ve test verileri değiştirilmeden yalnızca Xception'ın beklediği preprocess_input fonksiyonundan geçirilir. Amaç, veri çeşitliliğini artırırken kaş bölgesindeki ince doku, kıl sınırı ve lokal artefaktları aşırı dönüşümlerle bozmamaktır.

Augmentation	Ayar	Neden
Horizontal flip	Kapalı	Tek kaş veya sağ/sol semantiğini yanlış değiştirmek için.
Random rotation	factor=0.015	Çok küçük açısal değişime tolerans kazandırmak için.
Random zoom	± 0.03	Kaş ROI ölçeğindeki küçük değişimleri simüle etmek için.
Random contrast	0.03	Hafif kontrast farklarına dayanıklılık için.
Blur	Yok	İnce deepfake texture/kenar ipuçlarını silmemek için.
Random erasing	Yok	Lokal kaş artefaktlarını yapay biçimde yok etmemek için.

Augmentation sonrası preprocess_input uygulanır. Bu adım, piksel değerlerini Xception'ın ImageNet eğitiminde kullandığı girdi ölçeğine uygun hale getirir.

5.6. Cell 6 - Xception Modelinin Kurulması

Modelin omurgası TensorFlow Keras içindeki Xception'dır. weights='imagenet' kullanıldığı için model rastgele ağırlıklardan başlamaz; genel görsel örüntüleri daha önce öğrenmiş bir başlangıç noktasına sahiptir. include_top=False seçimiyle ImageNet'in orijinal 1000 sınıflı sınıflandırıcı başlığı çıkarılır.

Katman	Ayar	Görev
Xception backbone	ImageNet, include_top=False	Kaş ROI'den deep feature üretir.
GlobalAveragePooling2D	-	Spatial feature map'leri tek feature vektörüne indirir.
Dropout	0.30	Overfitting riskini azaltır.
Dense	256, ReLU	Kaş-deepfake ayrımı için öğrenilebilir ara temsil oluşturur.
Dropout	0.20	İkinci düzenleme katmanıdır.
Output Dense	1, sigmoid	P(fake) olasılığı üretir.

```

Input: 224×224×3 eyebrow ROI
↓
ImageNet pretrained Xception (include_top=False)
↓
GlobalAveragePooling2D
↓
Dropout(0.30)
↓
Dense(256, ReLU)
↓
Dropout(0.20)

```

↓
Dense(1, Sigmoid) = P(fake)

İlk aşamada xception_base.trainable=False olduğu için backbone ağırlıkları güncellenmez. Optimizer Adam, başlangıç learning rate 1e-4 ve loss BinaryCrossentropy'dir. İzlenen temel metrikler accuracy, precision, recall ve AUC'dir.

5.7. Cell 7 - Frozen Backbone ile İlk Eğitim

İlk gerçek model.fit çağrısı bu hücrede gerçekleşir. Xception backbone dondurulmuştur; dolayısıyla ImageNet'ten gelen convolution katmanları korunur ve esas olarak yeni eklenen sınıflandırıcı katmanlar kaş veri setine göre öğrenir.

Ayar	Değer
Maksimum epoch	15
Learning rate	1e-4
Model seçme metriği	val_auc
EarlyStopping	val_auc, patience=4, restore_best_weights=True
ReduceLROnPlateau	val_loss, factor=0.5, patience=2, min_lr=1e-7
Checkpoint	best_xception_frozen.keras
Log	frozen_training_log.csv + frozen_training_history.csv

Maksimum 15 epoch tanımlansa da EarlyStopping nedeniyle eğitim daha erken durabilir. ModelCheckpoint yalnızca validation AUC en iyi olduğunda modeli kaydeder. Bu sayede son epoch yerine validation açısından en iyi frozen model korunur.

5.8. Cell 8 - Xception Fine-Tuning

Fine-tuning aşaması frozen eğitimdeki en iyi modeli yükler. Xception backbone genel olarak tekrar trainable yapılır, ancak önce tüm katmanlar dondurulur ve yalnızca son 30 katman içinden BatchNormalization olmayanlar eğitime açılır.

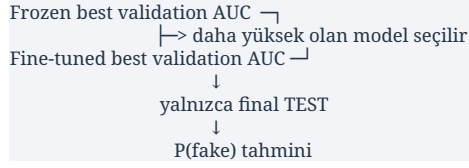
- UNFREEZE_LAST_N = 30
- BatchNormalization katmanları küçük dataset ve batch size nedeniyle frozen tutulur.
- Learning rate 1e-5'e düşürülür; bu, pretrained ağırlıkların sert biçimde bozulmasını önlemek için daha kontrollü bir adımdır.
- Model trainable katmanlar değiştiği için yeniden compile edilir.
- Fine-tuning maksimum 10 epoch sürer.
- ModelCheckpoint, EarlyStopping, ReduceLROnPlateau ve CSVLogger mekanizmaları bu aşamada da kullanılır.

Fine-tuning'in amacı

İlk aşamada Xception genel görsel feature'ları hazır biçimde kullanır. Fine-tuning'de son convolution bloklarının kaş bölgesindeki REAL/FAKE ayırma daha fazla adapte olması hedeflenir. Kod yorumlarında kaş-kıl/cilt sınırı, lokal texture, blending ve doğal olmayan kenar yapıları gibi olası örüntülere duyarlılığın artabileceği belirtilir; ancak bu tür özelliklerin gerçekten öğrenilip öğrenilmediğini doğrulamak için Grad-CAM benzeri açıklanabilirlik analizi gerekir.

5.9. Cell 9 - En İyi Model Seçimi ve Test Tahmini

Frozen ve fine-tuned eğitim özetleri okunur. İki aşama validation AUC açısından karşılaştırılır ve yüksek AUC'ye sahip model final test için seçilir. Bu yaklaşım bilimsel açıdan önemlidir çünkü test seti model seçme aracı olarak kullanılmaz.



Seçilen model test datasetinde predict() ile her görüntü için bir sigmoid skoru üretir. Bu skor P(fake) olarak yorumlanır. Varsayılan karar eşiği 0.50'dir: $P(fake) \geq 0.50$ ise FAKE, aksi halde REAL.

test_predictions.csv dosyasında görüntü yolu, gerçek etiket, gerçek sınıf, fake_probability, tahmin edilen etiket, tahmin edilen sınıf ve doğru/yanlış bilgisi saklanır.

5.10. Cell 10 - Final Test Metrikleri

Bu hücre modeli yeniden eğitmez. Cell 9'da üretilmiş y_true, y_pred ve test_probabilities değerlerini kullanarak final değerlendirme metriklerini hesaplar.

Metrik	Ne ölçer?
Accuracy	Tüm test örneklerinin ne kadarının doğru sınıflandırıldığını gösterir.
Balanced Accuracy	REAL ve FAKE sınıflarının recall değerlerini dengeli biçimde özetler.
Precision (FAKE)	FAKE denilen örneklerin ne kadarının gerçekten FAKE olduğunu gösterir.
Recall / Sensitivity	Gerçek FAKE örneklerin ne kadarının yakalandığını gösterir.
Specificity	Gerçek REAL örneklerin ne kadarının doğru REAL bulunduğunu gösterir.
F1 Score	FAKE precision ve recall arasındaki harmonik dengeyi gösterir.
ROC-AUC	Sabit bir eşikten bağımsız olarak REAL/FAKE sıralama kabiliyetini ölçer.
TN / FP / FN / TP	Confusion matrix'in dört temel hücresini verir.
FPR / FNR	Yanlış FAKE ve kaçırılan FAKE oranlarını verir.

Metrikler JSON ve CSV formatlarında kaydedilir; classification_report.txt ile sınıf bazlı precision, recall ve F1 raporu oluşturulur.

5.11. Cell 11 - Görselleştirme ve Rapor Figürleri

Bu hücre eğitim yapmaz; önceki aşamalarda üretilen tahminler ve training history dosyalarından rapor görselleri oluşturur. Grafikler PNG formatında 600 DPI çözünürlükle kaydedilir.

Dosya	İçerik
01_confusion_matrix.png	TN, FP, FN ve TP dağılımı
02_roc_curve.png	ROC eğrisi ve AUC
03_frozen_accuracy.png	Frozen train/validation accuracy eğrileri
04_frozen_auc.png	Frozen train/validation AUC eğrileri
05_frozen_loss.png	Frozen train/validation loss eğrileri
06_finetune_accuracy.png	Fine-tuning accuracy eğrileri
07_finetune_auc.png	Fine-tuning AUC eğrileri

08_finetune_loss.png	Fine-tuning loss eğrileri
09_frozen_vs_finetuned.png	Frozen ve fine-tuned validation sonuçlarının karşılaştırması
10_final_test_metrics.png	Final test metriklerinin bar chart özeti

5.12. Cell 12 - Final Artifacts, Run Summary ve Manifest

Son hücre deneyin arşivleme ve tekrarlanabilirlik katmanıdır. Seçilen checkpoint artifacts klasörüne Xception_Eyebrow_Final.keras adıyla kopyalanır ve deney ortamını açıklayan yardımcı dosyalar oluşturulur.

- model_summary.txt: seçilen modelin mimari özetini saklar.
- config_resolved.yaml: veri, eğitim, augmentation ve model seçimi parametrelerini tek yapılandırmada toplar.
- requirements_lock.txt: çalışma ortamındaki Python paketlerini kaydeder.
- environment.json: Python/platform/TensorFlow gibi ortam bilgilerini kaydeder.
- run_summary.json: dataset büyüklükleri, model stratejisi, seçilen model ve test metriklerini özetler.
- output_manifest.csv: run klasöründeki her dosyanın relative path, boyut ve SHA-256 hash bilgisini çıkarır.
- Final structure check: gerekli klasör ve kök dosyaların oluşturulduğunu ekrana raporlar.

6. Transfer Learning ve Fine-Tuning Mantiğı

Kodun en önemli makine öğrenmesi yaklaşımı iki aşamalı transfer learning stratejisidir.

6.1. Neden Xception sıfırdan eğitilmiyor?

weights='imagenet' ile başlayan Xception, görüntülerdeki genel kenar, doku, şekil ve daha karmaşık görsel örüntüler için önceden öğrenilmiş filtrelerle sahiptir. Kaş ROI dataseti sınırlı olduğunda bu başlangıç, rastgele ağırlıklardan tamamen sıfırdan eğitim yapmaya göre daha verimli bir başlangıç sağlar.

6.2. Frozen aşama ne öğrenir?

Backbone dondurulduğu için ImageNet feature extractor sabit kalır. Öğrenme ağırlıklı olarak GlobalAveragePooling sonrası eklenen Dense sınıflandırıcı bölümünde gerçekleşir. Böylece sistem önce 'hazır görsel feature'lardan REAL/FAKE ayrımı yapılabilir mi?' sorusuna cevap arar.

6.3. Fine-tuning ne ekler?

Fine-tuning'de Xception'ın son katmanlarının bir kısmı açılır. Bu aşamada model yalnızca classifier ağırlıklarını değil, deep feature extractor'ın son seviyelerini de kaş deepfake domain'ine göre hafifçe güncelleyebilir. Düşük learning rate kullanılması, pretrained bilgiyi tamamen bozmak yerine küçük ve kontrollü uyarlamalar yapılmasını amaçlar.

7. Kaş Bölgesine Özel Tasarım Kararları

Karar	Kaş açısından gerekçe
Hazır eyebrow ROI kullanımı	Modelin dikkatini tüm yüz yerine kaş/periocular lokal bölgeyle sınırlar.
Hafif augmentation	Kaş kılları, doku ve sınır geçişleri gibi ince ayrıntıları aşırı bozmamayı amaçlar.
Flip kapalı	Sağ/sol veya tek-kaş semantiği korunur.
Blur ve erasing yok	Potansiyel deepfake artefaktlarının yapay olarak silinmesi engellenir.
224×224 resize	Xception ile standart ve tutarlı girdi sağlar.
Fine-tuning düşük LR	Kaş bölgesine uyarlama yapılırken pretrained feature'ların korunmasını amaçlar.

8. Dayanıklılık ve Hata Toleransı

Notebook'un önceki sürümlere göre en kritik iyileştirmesi veri erişim hatalarına dayanıklı hale getirilmesidir.

- Gerekli Drive klasörlerinin tamamı eğitimden önce doğrulanır.
- Tüm kaş ROI dosyaları local cache'e kopyalanır.
- Her kopyalama işlemi hata halinde üç kez denenir.
- Kopyalanamayan dosyalar ayrı CSV loguna yazılır.
- Local cache'teki görüntüler PIL Image.verify ile okunabilirlik kontrolünden geçirilir.
- Eksik/bozuk görüntüler train/val/test listelerinden çıkarılır.
- Gerçek kullanılan veri sayıları doğrulama sonrası yeniden hesaplanır.
- Boş kalan bir split varsa eğitim durdurulur; sessizce hatalı deney yapılmaz.

Neden bu önemli?

Bir deep learning deneyi uzun sürebilir. Tek bir bozuk JPEG veya Drive bağlantısında tek bir anlık NotFoundError yüzünden 5., 10. veya 15. epoch'ta eğitimin tamamen çökmesi ciddi zaman kaybıdır. Bu notebook, problemi eğitimden önce görünür hale getirmeye ve mümkünse yalnızca problemlili dosyayı dışarıda bırakmaya çalışır.

9. Deneyisel Doğruluk ve Bilimsel Tasarım

9.1. Train, validation ve test rolleri

Split	Rol
Train	Model parametrelerini öğrenmek için kullanılır; augmentation yalnızca burada uygulanır.
Validation	Epoch boyunca model performansını izlemek ve frozen/fine-tuned model seçmek için kullanılır.
Test	Model seçimi tamamlandıktan sonra nihai, tarafsız performans değerlendirmesi için kullanılır.

Notebook'un model seçimini validation AUC üzerinden yapması, test setini optimizasyon kararlarından uzak tutar. Bu, test performansının daha tarafsız yorumlanabilmesi açısından doğru bir deney tasarımıdır.

9.2. Tekrarlanabilirlik

- SEED=42 Python, NumPy ve TensorFlow için sabitlenir.
- Dataset dosya listeleri sorted() ile deterministik sıraya alınır.
- Run ID zaman damgasıyla ayrılır; eski deneylerin üzerine yazılmaz.
- CSV training history dosyaları epoch bazında saklanır.
- Model/augmentation konfigürasyonları JSON/YAML olarak kaydedilir.
- Paket ve ortam bilgileri saklanır.
- Output manifest SHA-256 hash'leri dosya bütünlüğünü kontrol etmeyi kolaylaştırır.

10. Kodun Ürettiği Ana Dosyalar

Dosya	Amaç
best_xception_frozen.keras	Frozen aşamadaki en iyi val_auc checkpoint'i.
best_xception_finetuned.keras	Fine-tuning aşamadaki en iyi val_auc checkpoint'i.
Xception_Eyebrow_Final.keras	Model seçimi sonrası artifacts altına kopyalanan nihai model.
frozen_training_history.csv	Frozen epoch metrikleri.
finetune_training_history.csv	Fine-tuning epoch metrikleri.
model_selection_and_test_summary.json	Hangi modelin neden seçildiği ve Keras test değerlendirmesi.
test_predictions.csv	Örnek bazlı P(fake) ve tahmin kayıtları.
final_test_metrics.json / .csv	Final test metrikleri.
classification_report.txt	REAL/FAKE sınıf bazlı precision/recall/F1 özeti.
figure_manifest.json	Üretilen figürlerin kayıt listesi.
run_summary.json	Deneyin üst düzey özet kaydı.
output_manifest.csv	Tüm çıktı dosyaları + hash bilgisi.

11. Kodun Güçlü Yönleri

- Hazır pretrained Xception kullanarak küçük/orta ölçekli datasetlerde mantıklı bir transfer learning başlangıcı sağlar.
- Kaş ROI'sine göre augmentation'ı konservatif tutar.
- Drive erişim sorunlarına karşı local cache kullanır.
- Bozuk/eksik görüntüleri eğitimden önce doğrular ve loglar.
- Model seçimini test yerine validation AUC ile yapar.
- Frozen ve fine-tuned aşamaları ayrı ayrı checkpoint ve history dosyalarıyla kayıt altına alır.
- Final modeli, tahminleri, metrikleri, görselleri, config ve environment bilgisini tek bir run yapısında toplar.
- SHA-256 manifest ile çıktı bütünlüğünün izlenmesini kolaylaştırır.

12. Kodun Sınırları ve Gelecekte Eklenebilecekler

Aşağıdaki noktalar notebook'un mevcut kapsamı dışındadır; bunlar hata değil, deney kapsamı sınırlamalarıdır.

- Kaş ROI kalite kontrolü görsel/semantik açıdan otomatik puanlanmıyor; yalnızca dosya okunabilirliği kontrol ediliyor.
- Video-level prediction aggregation uygulanmıyor; test değerlendirmesi frame/ROI seviyesinde.
- Threshold sabit 0.50; validation setinden optimum threshold seçimi uygulanmıyor.
- Grad-CAM veya başka açıklanabilirlik yöntemi bulunmuyor.
- Cross-validation uygulanmıyor.
- Model farklı backbone'larla otomatik kıyaslanmıyor.
- Hand-crafted feature fusion bu notebook'ta yok.
- Split'in gerçekten video-bazlı olup olmadığı bu notebook içinde ayrıca denetlenmiyor.

13. Sonuç

Xception_Eyebrow_Deepfake_Colab_ROBUST_FIXED notebook'u, hazır kaş ROI görüntülerini kullanarak transfer learning ve fine-tuning tabanlı, uçtan uca izlenebilir bir deepfake sınıflandırma deneyi gerçekleştirir. Kodun temel yaklaşımı; ImageNet üzerinde önceden eğitilmiş Xception'ı önce sabit bir feature extractor olarak kullanmak, ardından son katmanları düşük öğrenme oranıyla kaş domain'ine uyarlamak ve nihai modeli validation AUC üzerinden seçmektir.

Teknik açıdan notebook yalnızca model eğitmekle kalmaz; veri erişim dayanıklılığı, bozuk dosya kontrolü, deney loglama, checkpoint yönetimi, test tahminlerinin saklanması, metrik üretimi, 600 DPI figür oluşturma ve bütün çıktıların manifest ile arşivlenmesi gibi deney yönetimi adımlarını da kapsar. Bu nedenle kod, tek seferlik bir eğitim scriptinden çok, tekrarlanabilir bir deney pipeline'ı olarak değerlendirilebilir.

Tek cümlelik açıklama

Bu kod, hazır kaş ROI frame'lerini güvenli biçimde okuyup ImageNet pretrained Xception ile REAL/FAKE sınıflandırması yapar; önce transfer learning, sonra fine-tuning uygular, en iyi modeli validation AUC ile seçer ve test sonuçlarını bütün artefaktlarıyla arşivler.

Ek A - Ana Hiperparametre Özeti

Parametre	Değer
Model	Xception
Pretrained weights	ImageNet
include_top	False
Input	224×224×3 RGB
Batch size	16
Sınıflar	REAL=0, FAKE=1
Frozen LR	1e-4
Frozen max epoch	15
Fine-tune last N	30
Fine-tune LR	1e-5
Fine-tune max epoch	10
Optimizer	Adam
Loss	BinaryCrossentropy
Model selection	Validation AUC
Decision threshold	0.50
Seed	42

Ek B - Basitleştirilmiş Pseudocode

```

mount_google_drive()

check_required_eyebrow_folders()
copy_eyebrow_dataset_to_local_cache(retry=3)

files = recursively_collect_images()
files = validate_images_and_log_invalid(files)

train_ds = build_tf_dataset(train, shuffle=True)
val_ds = build_tf_dataset(val, shuffle=False)
test_ds = build_tf_dataset(test, shuffle=False)

train_ds = light_eyebrow_augmentation(train_ds)
train_ds, val_ds, test_ds = xception_preprocess_input(...)

model = Xception(weights="imagenet", include_top=False)
freeze_backbone(model)
add_binary_classifier(model)

train_classifier(max_epochs=15, monitor="val_auc")

load_best_frozen_model()
unfreeze_last_layers(n=30, keep_batchnorm_frozen=True)
fine_tune(max_epochs=10, learning_rate=1e-5)

best_model = choose_by_validation_auc(frozen, finetuned)

probabilities = best_model.predict(test_ds)
predictions = probabilities >= 0.50

calculate_final_metrics()
create_600dpi_figures()
export_final_model_config_logs_predictions_summary_manifest()

```